2322ENG: Engineering C

Programming Laboratory 3: Loops

Instructions:

This lab will introduce one of most important tools available to programmers - loops. The three different types of loops in C will be introduced and explained in this lab, along with more practice of the things you have learned in previous weeks.

You should have read all "Operators, Flow Control and Data Types" notes and watched all of the associated videos prior to attending this lab!

Homework Question:

These questions should be completed *prior* to the lab session.

1. **Write C code which prints out the numbers 1 to 10 to the screen, 1 per line, using a WHILE loop, DO..WHILE loop, and FOR loop. 3 separate programs or one program with unused loops commented out is required.**
2. **Write a C program that prints out the first 20 powers of 2. For example, 1, 2, 4, 8, 16..**
3. **Write a C program that reads *characters* from the keyboard until a numerical digit ('0' to '9') is entered.**
4. **Write a C program which adds up all the numbers between 1 and 50. Use any type of loop.**
5. **Write a C program that prompts the user to enter a number, and then prints out the line "Hello" to the screen that many times.**
6. **Write a C program that continues asking the user to enter a number (integer) until they enter the number 7. Once that happens, the program should print "Goodbye" to the screen and terminate.**

Loops

Loops are one of the most useful of programming constructs, allowing programs to repeat a sequence of statements many times. C has three loop constructs and although they are all similar in nature, they have a few differences which make them more useful in particular situations.

In general, there are two reasons loops are used. The first is when you would like to repeat an operation a fixed number of times. For example, if you wanted to find the largest number from a list of 10 numbers. To do this, we need to compare each number in the list to the current largest number, and if it is larger, replace it. So, the same operation is repeated 10 times. For these type of tasks, the *for loop* is generally the most appropriate.

The second type of loop is when you aren't sure exactly how many times you need the statements run, but rather they should run until some condition is met. A very simple example of this would be a program which continues to add up numbers until a particular value is entered by the user (zero, negative, or some non-numerical value perhaps). In this case we don't know exactly how many times it will run, but we know the condition for it to terminate. In such cases, the *while loop* or *do..while* loops are generally the appropriate choice.

While Loops
The simplest of the C loop constructs is the *while* loop. This loop has a very simple syntax, but can still be used to do quite powerful operations. In fact, any loop can be rewritten as a *while* loop.

The syntax of the while loop is:

```
while ( condition ){      // loop guard
     statement1 ;
     statement2 ;         // loop body
     …
}
```

As you saw with *if* statements last week, you can also omit the braces from a *while* loop. However if this is done, only a single statement will be part of the loop. For this reason it is recommended that you always use the braces, even if there is only a single statement in the loop body.

The operation of the *while* loop is also quite straightforward. The *condition* is checked at the beginning of each iteration and evaluated as either TRUE or FALSE. If it is true, the loop body is executed, then the loop repeats from the start again, and the condition is checked again. If the condition is not true, the loop finishes and the program continues to the next statement. Note that because the condition is checked prior to the loop running, it is possible that the loop body does not run even once.

For example, the following program reads integer values from the user until zero is entered:

```
#include <stdio.h>

int main ( void ){

     int number = 1 ;  // note this must be set to a non-zero
                       // value to ensure the loop runs once

     while ( number != 0 ){  // loop continues until number == 0
          scanf ( "%d", &number ) ;
     }
     return ( 0 ) ;
}
```

1. **Enter the program above and check that it runs correctly.**
2. **Modify the loop so that the user can enter any negative number to stop the program, rather than 0.**

One important thing to note about this loop is that the controlling variable, *number*, is modified during the loop body. If this does not happen, then it is not possible for the loop to terminate. This is something that should be true for all loops (except 'infinite' loops), in order to terminate then it must be possible for the controlling variable(s) to be modified during the loop body. The loop must also be written in such a way that the variable is moving closer towards the termination condition, rather than away from it.

Do..while Loops
The second type of loop in C is very similar to the *while* loop. In fact the only difference is that in a *do..while* loop the loop guard (condition) is checked at the **end** of each loop body iteration, rather than the beginning. This has the important result that the loop body must be executed *at least once*. In the example above, for instance, using a *do..while* loop would mean that the *number* variable would not need to be assigned a non-zero value at the beginning, because the loop would always run once, and thus read a value from the user.

The syntax of the *do..while* loop is as follows:

```
do {
     statement1 ;
     statement2 ;
     …
} while ( condition );
```

The *do..while* loop is the most appropriate choice where your loop must always run at least once, regardless of the starting value of the controlling variable(s). For instance, consider a program that reads a non-negative integer from the user, then continually divides the number by two until it reaches zero, displaying each result to the screen. If the user enters zero, and a while loop is used, the loop would instantly terminate and nothing would display. Using a do..while loop ensures that the loop executes at least once, displaying zero to the screen before terminating. The code would be:

```
#include <stdio.h>

int main ( void ){

     int x ;

     printf ( "Enter a non-negative number: " ) ;
     scanf ( "%d", &x ) ;

     do{
          x = x / 2 ;     // divide number by 2
          printf ( "%d\n", x ) ;    // print to the screen
     } while ( x > 0 ) ;
}
```

3. **Enter and run the program above, ensuring that is works properly for all non-negative inputs including zero.**
4. **Modify the program to use a while loop instead, and note the different behaviour when zero is entered.**

For Loops
The last type of loop in C, and the most complex, is the *for* loop. This loop has a somewhat more complicated syntax, as follows:

```
for ( init ; condition ; increment ){
```

```
    statement1 ;
    statement2 ;
    …
}
```

The first thing you'll notice that's different about this loop is that there are three parts within the brackets, separated by semicolons. You can leave any or all of these three sections empty, however the semicolons must always be present.

The operation of the *for* loop is very similar to a *while* loop, with a few differences. The middle part of the statement, *condition*, is in fact identical to the condition of a *while* loop, and in the same way determines when the loop will terminate (when it becomes false). This condition is also tested at the beginning of each loop body iteration, just like the *while* loop. If the first and third parts of the *for* loop are omitted, the loop is in fact identical to a *while* loop.

The other two terms are what makes the *for* loop different. The first, the initialisation statement, runs once and once only, at the beginning of the first iteration, but before the loop guard is checked for the first time. This is typically used to setup a variable associated with the loop, but can be anything. The most common usage is to set a counting variable to 0 or other starting value.

The last statement in the *for* loop is known as the increment statement, because it is usually used to increment or decrement a variable. However it can be used for many other purposes as desired. The increment statement is run once per loop iteration, *after* the loop body.

The typical use of a *for* loop is to execute the loop a fixed number of times. In this case, the initialisation statement will set the counter to zero, the loop guard will check that it hasn't reached the desired number of iterations yet, and the increment term will increase the value of the counter by one. For example, to print out "Hello world" ten times, the following code would be used:

```
int i ;

for ( i = 0 ;i < 10 ; i++ ){
    printf ( "Hello world" ) ;
}
```

Variables called *i*, *j*, and *k* are traditionally used as loop counter variables, although it is better programming practice to use more meaningful names.

5. **Complete the missing lines of the following program, which reads ten floating point numbers from the user and calculates their average.**

```
#include <stdio.h>

int main ( void ){
   int i ;
   float number, total = 0, average ;

   for ( ???? ; ???? ; ???? ){
        scanf ( "%f", &number ) ;
```

```
        ???????
    }
    ???? ;
    printf ( "The average of the ten numbers is %f", ???? ) ;
}
```

As well as being used as a counter, the loop variable in a *for* loop can also be used in calculations, printed to the screen, etc. A very simple example of this would be to add up all the numbers between 1 and 100, while would be done by:

```
int i, total = 0 ;

for ( i = 1 ; i <= 100 ; i++ ){
     total += i ;   // add the value of i to the total
}
printf ( "The sum of 1 to 100 is %d\n", total ) ;
```

The maximum value of the loop does not have to be a fixed number, it can be a variable entered by the user or obtained from elsewhere.

6. **Modify the code above so that the program adds up the numbers from 1 to *x*, where *x* is entered by the user (prior to the loop beginning).**

Break and Continue

Within the body of a loop, the ***break*** statement may be used to terminate the loop instantly. This does not check the loop guard condition, it instantly stops the loop and execution of the program continues as normal. Whilst this can be useful, it is not considered good programming practice and should be used only when other methods of stopping the loop aren't practical.

The *continue* statement is similar, but rather than stopping the loop it only finishes the current iteration. The loop guard will then be checked to determine if the loop continues or not, and the loop proceeds as normal. Like *break*, this statement should be used sparingly as it is not considered good practice, but it can be useful in some situations.

Infinite Loops

Sometimes it desirable to have a loop that runs forever, for example on a microcontroller system that will never be turned off but should repeat a certain task forever. This is most easily accomplished with a *while* loop, where the loop guard is simply '1'. Since this will always be true, the loop will never terminate, except if the ***break*** statement is used within it. If your program has an infinite loop, either deliberately or accidentally, you can press CTRL-C to stop it.

Assessable questions:

You may be called to present one of these questions during the last hour of the session, and this presentation will count for 5% of your final grade for the course. You will be called to do two such presentations during the trimester, at random times. Note that you will need to explain your solution to the demonstrator and make changes to the code if requested to demonstrate your understanding. Each question should take less than 5 minutes to complete, and you may be asked a few simple questions while presenting. If you are not present in the laboratory when you are due to present, you

will receive 0 for this assessment item.

1. **Write a C program which continually prompts the user to enter an integer, only stopping when the number 0 is entered. Your program should then display how many numbers were entered (including the 0) as well as the total of *all* numbers entered.**

2. **Write a C program which prompts the user to enter an integer, and then displays all the factors of that integer. To do this, you will need to check every number from 1 up to the number entered and see if it is evenly divisible (remainder is zero). HINT: Use an if statement within the loop to check if the original number is divisible by the current loop counter, and if it is then display that number as a factor.**

3. **Write a C program which simulates a simple guessing game. The program should choose a random number between 1 and 100 as the 'secret' number before the game begins. The user should then be continually prompted to enter their guess. After each guess is entered, your program should inform the user whether the secret number is higher or lower, or if they guessed correctly. This should continue until they finally get the number right.**

   HINT: To create the random number, use the following code at the beginning of your main function. You will also need to #include <stdlib.h> and #include <time.h> at the top of the program.

   ```
   srand ( time ( NULL )) ;
   int secret_num = rand() % 100 + 1 ;
   ```

4. **Write a C program which prompts the user to enter a line of text, then counts the number of digits (0-9), upper case letters, and lower case letters that the user enters. Hint: continue to read individual characters from the keyboard until the newline is read, and process each separately.**

Extra practice questions (complete in your own time):

1. **At the beginning of this lab, you copied a program which continually divided a number by two until it reached zero. Modify this program so that instead of printing out the new value of the number at each step, it instead prints out the *remainder* of that division by 2. The number should still be divided by 2 at each step, only the value that is printed should be changed. If you do this correctly, you will notice that the numbers printed represent the binary form of this number, in reverse order.**

2. **Using a similar approach as question 1, write a C program which adds up all the digits of a number entered by the user. For example, if the user enters 5296, the result should be 22. You will use an almost identical loop to that used in Q2.**

3. **Write a C program which calculates the factorial (!) of a number entered by the user. A factorial is defined as the product of all numbers from 1 up to and including the number itself. For example, 5! = 1 x 2 x 3 x 4 x 5 = 120. Note that factorials above about 13 are too large to compute, so you may get errors.**